Comparing MLP, LSTM, and Transformer Policies for Robot Imitation Learning on PushT

Maryam Sharifzadeh and Navraj Kambo UBC EECE 568 Project Proposal

Statement of research proposal

We propose to study how the choice of neural network changes the quality of a robot policy learned from human demonstrations (imitation learning). Using the public PushT dataset (a pusher moves a T-shaped block onto a target), we will train three policies that map recent observations to actions: a feed-forward network (Multi-Layer Perceptrons, MLP), a recurrent neural network (Long short-term memory, LSTM), and a small Transformer. Our question is: does a longer observation history ($k > 1$), together with a model that can use it, give a better policy than an MLP that sees only the current state ($k = 1$)? We will compare the models by prediction error on held-out episodes and by task score when they control the pusher in simulation. We will also study how small action errors grow over time (compounding error), and we will share simulation videos and code.

Motivation and significance

Imitation learning lets a person teach a robot by showing it, without writing a controller or a reward function [3]. Its simplest form, behavior cloning, is supervised regression from observations to expert actions, and goes back to ALVINN [1]. Its known weakness is compounding error: small errors lead the learner to states the expert never visited, and errors grow with the task length [2]. In the worst case this growth is quadratic, although recent theory shows it is not always so [12].

The policy architecture matters. Implicit models often outperform explicit ones but need more compute [4]. A large study found that recurrent policies (which use history) can outperform feed-forward ones, and that results depend strongly on design choices [5]. Transformers that predict a chunk of future actions worked well for fine manipulation with little data [6], and Diffusion Policy reports strong results on PushT [7]. History also carries a risk: a model can use non-causal information (for example, the previous action), which looks good on training data but fails when the policy runs on its own [10].

These newer methods are complex. A student or engineer first needs a clear answer to a simpler question: how much do history and attention (LSTM and Transformer) [8, 9] help compared with an MLP, on the same data and with the same training budget?

Data to be used in the work and how it will be collected

We will use the PushT demonstrations from the Diffusion Policy project [7], in the public Hugging Face LeRobot copy [11] (`lerobot/pusht_keypoints`): 206 episodes and 25,650 steps, recorded at 10 steps per second. Each step gives the pusher position (2 values), the action (the pusher's target position, 2 values), and a 16-value environment state (we expect keypoints of the T-shaped block, and we will confirm this). A human operator recorded the demonstrations in a 2D simulator, so we need no new data, and the same simulator can run our policies and record videos. Data cleaning and preparation has four steps:

1. Check for missing or repeated steps and for very short episodes.
2. Normalize inputs and actions to zero mean and unit variance, using training data only.
3. Build samples: the last k observations ($k = 1, 2, 4, 8$; the first frame is repeated at the start of an episode) give the next H actions ($H = 1$ or 8; actions after the episode ends are ignored in the loss).
4. Split by episode, not by step, with 5-fold cross-validation (about 165 training and 41 test episodes per fold; 10% of the training episodes for validation). This keeps neighboring steps of one episode out of both sets, and gives more reliable results from a small dataset.

Method Used in the Research

Problem and loss. We treat imitation as supervised regression (behavior cloning). A policy π_θ reads the last k observations and predicts the next H actions. We train it to minimize

$$\mathscr{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{H}\sum_{j=0}^{H-1}\left\| a_{t_i+j} - \pi_\theta(o_{t_i-k+1},\ldots,o_{t_i})_j \right\|_2^2 + \lambda\|\theta\|_2^2$$

where o is the observation (the pusher position and the 16 block values), a is the demonstrated action, and λ is a weight-decay term. This is the regression idea from the course, with a neural network in place of a linear model [16]: the first term is the mean squared error (MSE), the second is the ridge penalty, and the rest adds history (as in BC-RNN [5]) and action chunking [6]. We train with Adam [13] optimization and Dropout [14] regularization technique.

We compare three models with a similar size (under 1 million parameters). **MLP (baseline).** The k observations are joined into one vector x and passed through two hidden layers, $z = \mathrm{ReLU}(W_2\,\mathrm{ReLU}(W_1 x + b_1) + b_2)$, and a linear layer outputs the H actions. It has no memory, so with $k = 1$ it reacts only to the current state. **LSTM [8].** The observations are read one by one, and gates decide what to keep and what to forget:

$$i_t,\, f_t,\, u_t = \sigma(W_{i,f,u}[x_t, h_{t-1}] + b_{i,f,u})$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c[x_t, h_{t-1}] + b_c), \qquad h_t = u_t \odot \tanh(c_t)\ [15]$$

The last hidden state goes to a linear layer that outputs the H actions. **Transformer [9].** Each observation becomes a token (a linear embedding plus position information). A small encoder (2 layers, 4 heads) uses attention,

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V\ [9]$$

where d is the key size. The last token is mapped to a chunk of H actions, as in [6].

Algorithm. For every model, fold and random seed (see Figure 1 on page 4):

1. Build the samples and train with Adam on the training episodes; keep the weights with the lowest validation loss (early stopping).
2. Offline test: compute the MSE on the test episodes.
3. Closed-loop test: in the simulator, give the last k observations to the policy, run its first predicted action, and predict again at the next step. Record the overlap score, the success rate and videos.

Experiments:

1. Offline error: mean squared error on the held-out test episodes, for the next action ($H = 1$) and for the 8-action chunk.
2. Closed-loop score (the main result): run each policy in the PushT simulator for 50 episodes from random starts, and report the average overlap of the T with the target (as given by the simulator) and the success rate. A lower offline error does not always mean a better policy [10].
3. Compounding error: let the policy feed on its own predictions and plot the error against the horizon. Behavior-cloning theory says the cost can grow with the square of the horizon [2].
4. Ablations: $k \in \{1,2,4,8\}$, $H \in \{1,8\}$, and dropout on or off, each with 3 random seeds (mean and standard deviation).

Compute. Python, PyTorch and Jupyter notebooks, as in the course, with an open-source PushT simulator (gym-pusht from LeRobot [11]). The dataset and the models are small (under 1 million parameters and about 25,000 samples), so a free Google Colab GPU or a laptop is enough.

Bibliography

[1] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 1, 1989. https://proceedings.neurips.cc/paper/1988/file/812b4ba287f5ee0bc9d43bbf5bbe87fb-Paper.pdf

[2] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. Int. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2011. https://proceedings.mlr.press/v15/ross11a/ross11a.pdf

[3] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, and J. Peters, "An algorithmic perspective on imitation learning," *Foundations and Trends in Robotics arXiv:1811.06711*, vol. 7, no. 1–2, 2018. https://arxiv.org/abs/1811.06711

[4] P. Florence et al., "Implicit behavioral cloning," in *Proc. Conf. on Robot Learning (CoRL)*, 2021. arXiv:2109.00137. https://arxiv.org/pdf/2109.00137

[5] A. Mandlekar et al., "What matters in learning from offline human demonstrations for robot manipulation," in *Proc. CoRL*, 2021. arXiv:2108.03298. https://arxiv.org/pdf/2108.03298

[6] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Robotics: Science and Systems (RSS)*, 2023. arXiv:2304.13705. https://arxiv.org/pdf/2304.13705

[7] C. Chi et al., "Diffusion Policy: Visuomotor policy learning via action diffusion," *Int. Journal of Robotics Research*, 2024. arXiv:2303.04137. https://arxiv.org/pdf/2303.04137

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. https://www.researchgate.net/publication/13853244_Long_Short-Term_Memory

[9] A. Vaswani et al., "Attention is all you need," in *NeurIPS*, 2017. arXiv:1706.03762. https://arxiv.org/pdf/1706.03762

[10] P. de Haan, D. Jayaraman, and S. Levine, "Causal confusion in imitation learning," in *NeurIPS*, 2019. arXiv:1905.11979. https://arxiv.org/pdf/1905.11979

[11] R. Cadene et al., "LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch," Hugging Face, 2024. https://github.com/huggingface/lerobot.

[12] D. J. Foster, A. Block, and D. Misra, "Is behavior cloning all you need? Understanding horizon in imitation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. https://proceedings.neurips.cc/paper_files/paper/2024/file/da84e39ae51fd26bb5110d9659c06e13-Paper-Conference.pdf.

[13] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980. https://arxiv.org/abs/1412.6980

[14] S. Hahn and H. Choi, "Understanding dropout as an optimization trick," arXiv:1806.09783, 2018. https://arxiv.org/pdf/1806.09783

[15] M. Brenndoerfer, "LSTM Gate Equations: Math NumPy Implementation," in *mbrenndoerfer.com* https://mbrenndoerfer.com/writing/lstm-gate-equations-mathematical-guide-implementation

[16] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. https://www.deeplearningbook.org

Use of AI. We used Claude (Anthropic) to check this proposal against the course rubric and to help edit its wording, structure and references. The authors reviewed all content and are responsible for it.

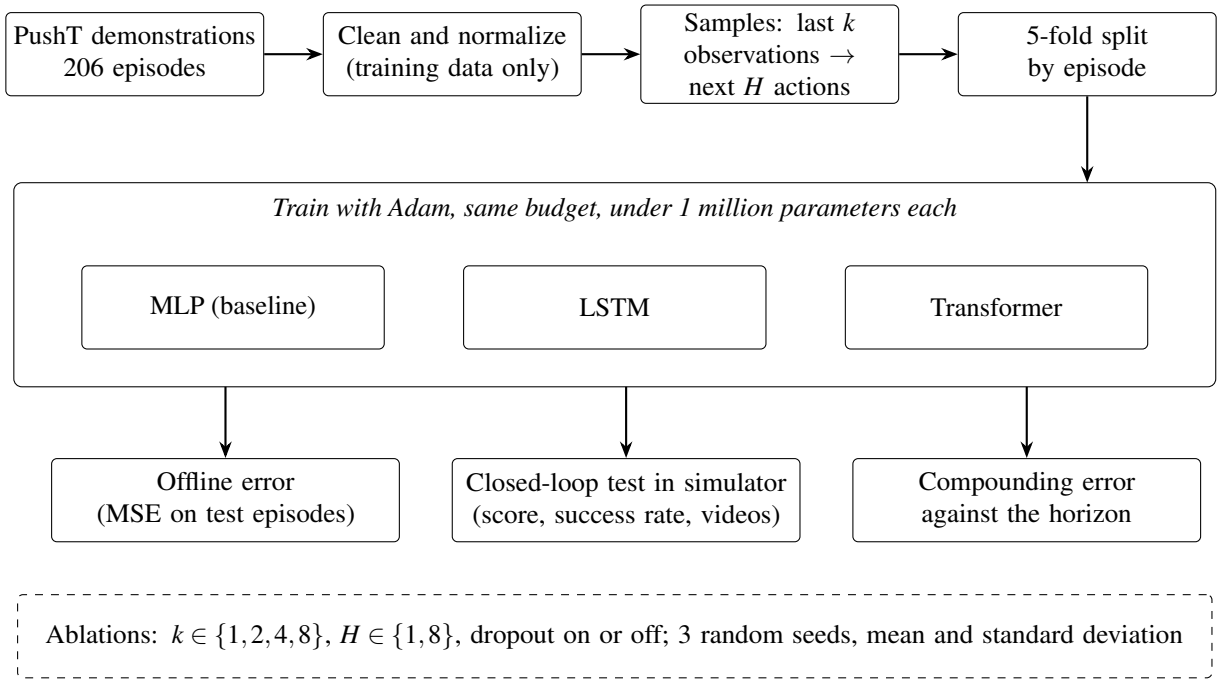


Figure 1: Pipeline of the study. The same cleaned and split data is used for three models. Each model is tested offline and in the PushT simulator, and the ablations vary the history length k, the chunk size H and dropout.